



Universidad de Valladolid

**Escuela de Ingeniería Informática de
Valladolid**

TRABAJO FIN DE GRADO

Grado en Ingeniería Informática

Mención en Ingeniería de Software

Título del TFG

Autor:

D. Sergio Alcón González

Tutores:

**D. Luis Ignacio Jiménez Gil
D. Mario Corrales Astorgano**

Resumen

Palabras clave:

Abstract

Keywords: ...

Índice general

1	Introducción	5
1.1	Contexto y Motivaciones	5
1.2	Objetivos	6
1.3	Estructura del documento	6
2	Metodología	7
2.1	Iteraciones del desarrollo	8
2.1.1	Iteración 1: Implementación del juego base	8
2.1.2	Iteración 2: Implementación de la inteligencia del bot	9
2.1.3	Iteración 3: Desarrollo de la interfaz de usuario	9
2.1.4	Iteración 4: Mejoras audiovisuales y pulido final	10
3	Antecedentes y Estado del Arte	11
3.1	Evolución de los juegos de estrategia	11
3.2	El juego original: <i>Stratego</i>	13
3.3	Variaciones, adaptaciones y videojuegos relacionados	14
3.4	Motor de videojuegos utilizado	15
4	Concepto del videojuego	23
4.1	Resumen del juego	23
4.2	Mecánicas	24
4.2.1	Tablero	24
4.2.2	Colocación inicial	24
4.2.3	Piezas	25

4.2.4	Turnos	26
4.2.5	Resolución de combates	27
4.2.6	Condiciones de victoria	29
4.2.7	Interacción y control	29
4.3	Ambientación	30
4.4	Diseño de niveles	32
4.5	Audiencia - Población objetivo	32
5	Planificación	33
5.1	Riesgos	33
5.2	Restricciones	34
5.3	Herramientas	35
5.4	Planificación inicial	36
5.5	Cronograma del proyecto	38
5.6	Variación de planificación	40
5.7	Costes del proyecto	40
6	Análisis	43
6.1	Actores y roles	43
6.2	Modelado de Requisitos	43
6.3	Casos de Uso	45
6.4	Modelo de dominio	45
6.5	Modelo de proceso de negocio	45
7	Diseño	47
7.1	Arquitectura del sistema	47
7.2	Módulos o Componentes del sistema	47
7.3	Diseño de clases	47
7.4	Diseño de interfaz de usuario	47
7.5	Patrones de diseño aplicados	47
7.6	Seguridad y rendimiento	47

8 Implementación	49
8.1 Organización del código	49
8.2 Licencias requeridas	49
8.3 Batería de pruebas	49
9 Conclusiones y Líneas futuras	51
A Manual de instalación	53
B Manual de usuario	55
C	57
D	59
Bibliografía	61

Índice de figuras

3.1	Tablero del Juego Real de Ur [1]	18
3.2	Tablero del juego Senet [2]	18
3.3	Tablero del juego Dou Shou Qi [3]	19
3.4	Tablero del juego L'attaque [4]	20
3.5	Tablero del juego Luzhanqi [5]	21
4.1	Esquema tablero	25
5.1	Diagrama de Gantt general del desarrollo del TFG, incluyendo todas las iteraciones y la documentación	39

Índice de tablas

3.1	Comparativa entre motores de desarrollo de videojuegos	17
5.1	Riesgos identificados y planes de contingencia actualizados	34
5.2	Herramientas	36
5.3	Fechas estimadas de finalización de las iteraciones del proyecto	37

Agradecimientos

Glosario

ACRONIMO Significado del acrónimo, del inglés *Si precediera del inglés*.

Capítulo 1

Introducción

1.1. Contexto y Motivaciones

1.2. Objetivos

El objetivo principal de este TFG es el diseño y desarrollo de un videojuego completamente funcional utilizando el motor de desarrollo Godot y el lenguaje de programación C#. El proyecto tiene como finalidad implementar un sistema completo de juego que incluya las mecánicas básicas, un oponente controlado por inteligencia artificial y una interfaz de usuario que permita al jugador interactuar de forma clara e intuitiva con el sistema.

Para alcanzar este objetivo general, se han definido los siguientes objetivos específicos:

- Diseñar e implementar la lógica fundamental del videojuego, incluyendo la estructura del tablero, la definición de las piezas y las reglas de movimiento y combate entre ellas.
- Desarrollar un sistema de control de turnos y gestión del estado del juego que permita ejecutar partidas completas respetando las reglas definidas.
- Implementar un bot inteligente capaz de analizar el estado del tablero y seleccionar movimientos válidos durante la partida.
- Diseñar e implementar una interfaz de usuario clara e intuitiva que proporcione al jugador información visual sobre el estado del juego y permita interactuar con el sistema de manera sencilla.
- Incorporar elementos audiovisuales, como animaciones y efectos sonoros, que mejoren la experiencia de juego y aporten mayor claridad a las acciones realizadas durante la partida.
- Adquirir experiencia práctica en el uso del motor de videojuegos Godot y en el desarrollo de sistemas interactivos utilizando el lenguaje de programación C#.

1.3. Estructura del documento

Este documento se organiza en distintos capítulos, que dividen el proyecto en ...

Metodología

En este capítulo se describe la metodología seguida para el desarrollo del proyecto, detallando la organización del trabajo y el proceso de elaboración, revisión y retroalimentación que ha guiado la evolución tanto del documento como del videojuego desarrollado. La metodología adoptada ha condicionado de forma directa las decisiones de diseño, la planificación de tareas y el alcance final del proyecto.

El desarrollo del proyecto *NeuroForge* se ha llevado a cabo siguiendo una metodología de carácter iterativo [7], basada en la elaboración progresiva de versiones parciales que han sido evaluadas y pulidas de manera continua. Este enfoque se basa en la idea de que el proyecto evoluciona mediante ciclos sucesivos de trabajo, revisión y mejora, en los que cada iteración incorpora los aprendizajes y correcciones surgidos a partir de la anterior.

A diferencia de las metodologías clásicas de desarrollo de software orientadas a equipos numerosos y ciclos de producción cerrados, el presente proyecto se ha desarrollado en un contexto académico individual, lo que ha requerido un planteamiento metodológico más flexible. En este sentido, el trabajo se ha estructurado en una secuencia de entregas parciales en las que se han presentado avances tanto a nivel conceptual como técnico. Cada una de estas entregas ha sido revisada por los tutores del proyecto, quien ha proporcionado observaciones, correcciones y sugerencias que han servido como base para orientar el trabajo de la siguiente iteración.

Este proceso de revisión continua ha permitido una mejora progresiva del proyecto, ya que cada nueva versión del documento y del videojuego ha incorporado ajustes y ampliaciones respecto a la anterior. Además, la retroalimentación temprana ha facilitado la detección de errores conceptuales, problemas de diseño o decisiones técnicas poco adecuadas en fases iniciales del desarrollo, evitando así que dichos problemas se acumulen a etapas más avanzadas y complejas.

La metodología iterativa ha favorecido una adaptación constante al alcance real del Trabajo de Fin de Grado. A lo largo del desarrollo se han tenido en cuenta las limitaciones de tiempo, recursos y complejidad propias de un proyecto académico individual, permitiendo reajustar objetivos y prioridades de forma gradual. Este enfoque ha resultado especialmente relevante en el diseño del videojuego, donde las mecánicas, sistemas y elementos visuales han sido definidos, evaluados y

refinados conforme se analizaba su viabilidad técnica y su coherencia con los objetivos del proyecto.

En la práctica, el desarrollo de *NeuroForge* ha seguido una progresión estructurada que ha comenzado con la concepción de la idea y el diseño inicial del videojuego, pasando por la planificación del trabajo, el análisis de riesgos y la definición de requisitos, hasta culminar en la implementación de una versión completa y funcional. Esta evolución gradual ha permitido mantener un control continuo sobre el estado del proyecto y asegurar la coherencia entre los objetivos planteados inicialmente y los resultados obtenidos.

2.1. Iteraciones del desarrollo

Con el objetivo de organizar el trabajo de forma progresiva y controlada, el desarrollo del proyecto se estructuró en varias iteraciones. Cada iteración se centró en la implementación de un conjunto concreto de funcionalidades del videojuego, permitiendo validar el funcionamiento del sistema antes de avanzar hacia etapas de mayor complejidad o con funcionalidades dependientes de las de anteriores iteraciones.

2.1.1. Iteración 1: Implementación del juego base

La primera iteración tuvo como objetivo el desarrollo de una versión básica y completamente funcional del videojuego. En esta fase se implementaron las mecánicas fundamentales necesarias para que una partida pudiera desarrollarse correctamente sin necesidad de un bot, estableciendo las bases estructurales del sistema.

Durante esta iteración se definió en primer lugar la estructura del tablero y de las piezas que intervienen en el juego, lo que permitió establecer el marco sobre el que se desarrollaría el resto de la lógica del sistema. A continuación, se implementó el sistema de movimiento de las piezas, incluyendo las restricciones y reglas que determinan cómo pueden desplazarse dentro del tablero.

Una vez establecido el sistema de movimiento, se desarrolló el sistema de combate entre piezas, encargado de gestionar las interacciones que se producen cuando dos piezas entran en conflicto dentro del tablero. Paralelamente, se implementó el sistema de gestión del estado del tablero, que permite actualizar de forma correcta la posición de las piezas tras cada acción realizada por los jugadores.

Asimismo, se incorporó el sistema de turnos que regula la alternancia entre jugadores durante la partida. Finalmente, se configuró el estado inicial del juego al comenzar una nueva partida y se implementaron las condiciones que determinan la finalización de la misma. Durante todo este proceso se realizaron pruebas funcionales básicas con el objetivo de verificar el correcto funcionamiento de las mecánicas implementadas.

Al finalizar esta iteración se obtuvo una primera versión completamente jugable del juego, que incluía todas las mecánicas esenciales necesarias para el desarrollo de una partida.

2.1.2. Iteración 2: Implementación de la inteligencia del bot

Una vez establecida la base funcional del videojuego, la segunda iteración se centró en el desarrollo de un sistema de inteligencia artificial que permitiera controlar un oponente automático.

El objetivo principal de esta fase fue dotar al juego de un adversario capaz de analizar el estado del tablero y tomar decisiones de forma autónoma, respetando las reglas establecidas en la iteración anterior. Para ello, el desarrollo de la inteligencia del bot se planteó de forma progresiva, comenzando con un sistema de toma de decisiones básico y evolucionando posteriormente hacia un comportamiento más avanzado.

En una primera etapa se diseñó un bot capaz de identificar los movimientos válidos disponibles en cada turno. A partir de este conjunto de acciones posibles, el sistema selecciona una de ellas de forma parcialmente aleatoria. Este enfoque permite garantizar que el bot respeta las reglas del juego y es capaz de participar correctamente en una partida, aunque sus decisiones no sigan todavía una estrategia compleja.

Una vez implementado este comportamiento inicial, se plantea la evolución del sistema hacia un modelo de decisión más avanzado basado en el aprendizaje mediante repetición. En este enfoque, el bot puede entrenarse jugando múltiples partidas contra sí mismo, lo que permite ajustar progresivamente su comportamiento y mejorar la calidad de sus decisiones a lo largo del tiempo.

Durante esta iteración también se integró el sistema de inteligencia artificial dentro del sistema de turnos del juego, permitiendo que el bot pueda actuar automáticamente cuando corresponde su turno. Asimismo, se realizaron distintas pruebas de funcionamiento para analizar su comportamiento en diversas situaciones de partida y comprobar su correcta integración con el resto de mecánicas del sistema.

Tras completar esta iteración, el videojuego pasó a contar con una experiencia de juego funcional para un solo jugador, sentando además las bases para futuras mejoras en el comportamiento del bot.

2.1.3. Iteración 3: Desarrollo de la interfaz de usuario

La tercera iteración estuvo orientada a mejorar la interacción entre el jugador y el sistema mediante el desarrollo de una interfaz de usuario más clara e intuitiva.

Durante esta fase se comenzó con el proceso de diseño de la interfaz mediante la elaboración de distintos bocetos y propuestas visuales. Estos bocetos permitieron definir la disposición de los elementos principales de la pantalla, como el tablero, las piezas, los indicadores de turno y los diferentes elementos de interacción con el jugador. Este proceso de diseño previo facilitó la planificación de la interfaz antes de su implementación dentro del juego.

A partir de este diseño inicial se trabajó en la representación visual del tablero y en la representación gráfica de las piezas, con el objetivo de facilitar la comprensión del estado del

juego. Sobre esta base visual se implementaron distintos indicadores que permiten al jugador identificar de forma clara qué piezas están seleccionadas y cuáles son los movimientos disponibles en cada momento.

Posteriormente, se desarrollaron distintos elementos de interacción que permiten gestionar la partida de manera más cómoda, como menús básicos para iniciar o reiniciar el juego. Paralelamente, se introdujeron mejoras en la visualización del estado de la partida, permitiendo mostrar información relevante como el turno actual o el resultado de determinadas acciones dentro del juego.

Con estas mejoras, esta iteración permitió aumentar significativamente la claridad visual del sistema y facilitar la interacción del usuario con el juego.

2.1.4. Iteración 4: Mejoras audiovisuales y pulido final

La última iteración se centró en la incorporación de mejoras visuales y sonoras, así como en la optimización general del sistema con el objetivo de obtener una versión final más completa y estable del videojuego.

Durante esta fase se añadieron efectos de sonido asociados a distintas acciones del juego, como movimientos o interacciones entre piezas, lo que contribuyó a mejorar la inmersión del jugador en la experiencia de juego. Además, se realizaron diversas mejoras visuales en el tablero y en la representación de las piezas, refinando su apariencia y coherencia estética.

Asimismo, se incorporaron animaciones asociadas a determinadas acciones del juego, como el desplazamiento de las piezas o las interacciones entre ellas. Estas animaciones permiten representar de forma más clara los cambios que se producen en el tablero y aportan mayor dinamismo a la experiencia de juego.

También se llevaron a cabo tareas de optimización del rendimiento del sistema con el fin de garantizar un funcionamiento fluido del videojuego. Finalmente, se realizaron pruebas adicionales destinadas a detectar posibles errores o comportamientos inesperados, lo que permitió corregir fallos y asegurar la estabilidad del sistema.

Esta última iteración permitió consolidar todos los elementos desarrollados en fases anteriores y obtener una versión final del videojuego preparada para su presentación en el contexto del TFG.

Antecedentes y Estado del Arte

En este capítulo se analizan los antecedentes y el estado del arte relacionados con el proyecto *NeuroForge*. Para ello, se realiza un recorrido por la evolución de los juegos de estrategia desde sus orígenes en los juegos de mesa clásicos hasta sus adaptaciones modernas, tanto físicas como digitales. Este análisis permite identificar las mecánicas fundamentales que han influido en el diseño del proyecto, así como contextualizar las decisiones adoptadas durante su desarrollo.

3.1. Evolución de los juegos de estrategia

Desde la antigüedad, los juegos de mesa han servido como una representación lúdica del conflicto, la planificación y la toma de decisiones. En las primeras civilizaciones, estos juegos cumplían una función tanto recreativa como cultural, y sentaron las bases de conceptos estratégicos que perduran hasta la actualidad.

Entre los ejemplos más antiguos se encuentra el *Juego Real de Ur*, originario de Iraq (aprox. del 2600-2400 a.C.) [8], presentaba un tablero dividido en secciones conectadas, tal y como se muestra en la Figura 3.1. Este diseño introducía decisiones tácticas relacionadas con el avance y la protección de las piezas, reforzando la importancia de la planificación a medio plazo. Ambos juegos, pese a su dependencia parcial del azar, contribuyeron a establecer una base conceptual sobre la que evolucionarían juegos de estrategia más complejos.

Aunque no se ha conservado un reglamento completo que permita reconstruir con total precisión las normas del juego, sí existe una descripción parcial recogida en una tablilla cuneiforme de origen babilonio, escrita entre los años 177 y 176 a.C. por el escriba Itti-Marduk-Balāṭu [1]. A partir de este testimonio y de los tableros conservados, se considera que el Juego Real de Ur consistía en una competición entre dos jugadores en forma de carrera a lo largo del tablero, con mecánicas comparables a juegos actuales como el parchís o el backgammon.

De forma similar, el juego egipcio *Senet*, documentado arqueológicamente durante el Imperio Nuevo, con ejemplares datados en el reinado de Amenhotep III (aprox. 1390–1353 a.C.) [2],

se desarrollaba sobre un tablero de treinta casillas, como se observa en la Figura 3.2. El desplazamiento de las piezas se determinaba mediante el lanzamiento de palos de conteo o huesos, introduciendo un componente de azar en la cantidad de casillas que podían avanzarse en cada turno. No obstante, el jugador conservaba un grado significativo de control estratégico, ya que podía decidir qué pieza mover, bloquear el avance del oponente o forzar retrocesos en determinadas situaciones. Esta combinación de aleatoriedad en la generación de movimientos y toma de decisiones tácticas en su aplicación convierte a *Senet* en uno de los primeros ejemplos conocidos de juego que equilibra azar y planificación. Aunque su mecánica era relativamente simple, introducía ya la idea de progresión de piezas y competición directa entre jugadores, aspectos clave en el desarrollo temprano del pensamiento estratégico [9].

Ambos juegos, pese a su dependencia parcial del azar, contribuyeron a establecer una base conceptual sobre la que evolucionarían juegos de estrategia más complejos.

Con el avance del tiempo, los juegos de estrategia comenzaron a abandonar progresivamente la dependencia del azar para centrarse en la planificación deliberada y la toma de decisiones informadas. En este contexto surge el *Ajedrez*, cuyo origen se sitúa en la India alrededor del siglo VI d.C., a partir del juego conocido como *Chaturanga* [10]. A diferencia de los juegos antiguos basados en carreras o desplazamientos condicionados por el azar, el ajedrez introduce una jerarquía claramente definida de piezas con capacidades diferenciadas y un sistema de captura directa. Estas características refuerzan conceptos como el sacrificio estratégico, el control del espacio y la planificación táctica a largo plazo. Asimismo, el ajedrez se caracteriza por ofrecer información completa a ambos jugadores, lo que lo convierte en un paradigma de la estrategia basada exclusivamente en la anticipación y el cálculo.

Dentro de esta evolución resulta especialmente relevante el juego chino *Dou Shou Qi*, también conocido como *Selva* o *Ajedrez animal*. A diferencia de juegos clásicos cuya antigüedad está bien documentada, el origen histórico de *Dou Shou Qi* no se encuentra claramente establecido. Las fuentes disponibles lo describen como un juego tradicional chino desarrollado con posterioridad al *Xiangqi*, del que probablemente deriva a nivel conceptual [3].

El juego se estructura en torno a una jerarquía asimétrica de unidades representadas por animales, cada uno con un rango definido, y emplea un sistema de captura por desplazamiento similar al de los juegos de ajedrez (Tablero en la Figura 3.3). No obstante, introduce excepciones explícitas en las reglas de captura —como la capacidad de una pieza de rango inferior para derrotar a otra superior bajo determinadas condiciones—, lo que rompe la jerarquía estricta y añade una dimensión estratégica basada en la gestión del riesgo y la anticipación del comportamiento del oponente. Este tipo de jerarquía no lineal anticipa principios fundamentales que aparecerán más adelante en los juegos de estrategia militar modernos.

El siguiente paso se produce a comienzos del siglo XX con la aparición de *L'Attaque*, un juego de mesa diseñado por Hermance Edan, quien registró su patente el 26 de noviembre de 1908 [11]. En su descripción original, el juego se define como un «*jeu de bataille avec des pièces mobiles sur damier*», es decir, un juego de batalla con piezas móviles sobre un tablero cuadriculado (Figura 3.4). La patente fue publicada oficialmente en 1909 por la Oficina de Patentes francesa, y el juego fue comercializado de forma continuada desde ese año hasta comienzos de la década de 1930.

L'Attaque introduce por primera vez de manera explícita la ocultación de información como mecánica central del diseño. Cada jugador dispone de un conjunto de piezas con valores numéricos jerárquicos, cuyo rango permanece oculto al oponente hasta el momento de la confrontación directa. El resultado del combate se resuelve comparando los valores de las piezas implicadas, eliminándose generalmente la de menor rango. La partida concluye cuando uno de los jugadores localiza y captura la pieza objetivo del adversario, identificada como la bandera.

Estas mecánicas suponen una ruptura clara con los juegos de información completa predominantes hasta ese momento, al trasladar una parte sustancial de la toma de decisiones al ámbito de la inferencia, el engaño y la gestión del riesgo. Asimismo, *L'Attaque* consolida una jerarquía militar explícita y permite la configuración libre de la disposición inicial de las unidades, lo que incrementa la profundidad estratégica desde el primer turno y refuerza la importancia de la planificación previa. En conjunto, estas innovaciones marcan un punto de inflexión en el diseño de juegos de estrategia, al introducir por primera vez de forma sistemática la incertidumbre como recurso estratégico.

La progresión histórica desde los primeros juegos de mesa hasta títulos como *L'Attaque* pone de manifiesto una evolución gradual desde mecánicas dominadas por el azar y el movimiento lineal hacia sistemas estratégicos cada vez más complejos, basados en la jerarquía de unidades, la confrontación directa y, finalmente, la ocultación de información. Esta acumulación progresiva de conceptos estratégicos, planificación, gestión del riesgo, engaño e inferencia, configura el marco teórico sobre el que se desarrollaría posteriormente *Stratego* y, por extensión, fundamenta las decisiones de diseño del proyecto *NeuroForge*.

3.2. El juego original: *Stratego*

El juego *Stratego* constituye uno de los referentes más influyentes dentro del ámbito de los juegos de estrategia con información imperfecta. Su diseño no surge de forma aislada, sino que se apoya en una serie de innovaciones mecánicas desarrolladas a comienzos del siglo XX, especialmente a partir del juego europeo *L'Attaque*, considerado su antecedente directo.

La versión moderna de *Stratego* fue desarrollada tras la Segunda Guerra Mundial por la compañía neerlandesa Jumbo, que adoptó una estética de inspiración militar napoleónica. El juego fue registrado comercialmente en 1960 y su difusión internacional se consolidó en 1961, cuando Milton Bradley obtuvo la licencia para su distribución en Estados Unidos [12].

Desde el punto de vista mecánico, *Stratego* se articula en torno a una jerarquía militar explícita de unidades, cuyos rangos permanecen ocultos al oponente hasta el momento del enfrentamiento directo. El sistema de combate se resuelve mediante la comparación de los valores de las piezas implicadas, eliminándose generalmente la de menor rango. El objetivo principal de la partida consiste en localizar y capturar la pieza objetivo del adversario, identificada como la bandera.

Una de las decisiones de diseño más relevantes es la configuración inicial libre del despliegue de las unidades, que traslada una parte sustancial del peso estratégico al momento previo al inicio de la partida. Esta característica, combinada con la ocultación de información, fomenta el engaño, la

deducción y la gestión del riesgo como elementos centrales de la experiencia de juego.

Desde el punto de vista material, *Stratego* ha experimentado diversas adaptaciones físicas a lo largo del tiempo. Las primeras ediciones utilizaban piezas de cartón impreso, que fueron sustituidas posteriormente por piezas de madera pintada. A partir de finales de la década de 1960, estas fueron reemplazadas por piezas de plástico, más económicas y estables. Este cambio no solo respondió a criterios de producción, sino que también redujo el riesgo de vuelco accidental de las piezas, un problema crítico en un juego donde la revelación involuntaria de la información compromete el desarrollo estratégico de la partida.

Desde una perspectiva de diseño, *Stratego* se caracteriza por la ausencia total de elementos aleatorios durante la partida. Todas las decisiones se basan en la planificación, la inferencia y la lectura del comportamiento del oponente, lo que lo convierte en un paradigma de la estrategia bajo información imperfecta. Estas características explican su influencia duradera y justifican su elección como principal referencia conceptual y base del diseño para el desarrollo del proyecto **NeuroForge**.

3.3. Variaciones, adaptaciones y videojuegos relacionados

Además de sus múltiples ediciones físicas, *Stratego* ha dado lugar a numerosas variantes temáticas ambientadas en universos de ficción como *Star Wars*, *El Señor de los Anillos*, o *Marvel*. Estas adaptaciones mantienen la estructura básica del juego original, introduciendo cambios estéticos y ajustes menores en las reglas según la ambientación.

En el ámbito digital, *Stratego* cuenta con versiones en línea que permiten partidas a distancia entre jugadores de todo el mundo como el *Stratego Online*¹ y apps móviles disponibles en iOS y Android. Estas plataformas incorporan modos clásicos y variantes con tableros alternativos o un número reducido de piezas, contribuyendo a mantener vigente el interés por el juego. El juego conserva además presencia activa en el ámbito competitivo, especialmente en países como Países Bajos, Alemania y Bélgica, incluyendo torneos oficiales y rankings europeos.

Junto a *Stratego*, existen otros juegos de mesa y digitales basados en principios similares. Un ejemplo es *Luzhanqi* [5], originario de China, cuyo tablero se muestra en la Figura 3.5. Este juego comparte la mecánica de piezas ocultas y configuración libre inicial, evidenciando una convergencia de ideas estratégicas entre distintas tradiciones culturales.

En el ámbito de los videojuegos, destacan títulos de estrategia por turnos que enfatizan la planificación, la jerarquía de unidades y la toma de decisiones en entornos discretos. Entre ellos se encuentran la saga *Advance Wars* (Intelligent Systems, Advance Wars series, Nintendo) y *Into the Breach* (Subset Games, Into the Breach). En ambos casos, aunque la información sobre el enemigo es visible, el peso estratégico recae en la gestión de unidades, el posicionamiento y la anticipación de las acciones del rival.

Dentro del ámbito de los videojuegos de estrategia por turnos que comparten principios

¹<https://www.stratego.com>

mecánicos con *Stratego*, destacan títulos como la saga *Advance Wars*. Esta serie, desarrollada por Intelligent Systems para plataformas de Nintendo entre 2001 y 2019, se basa en combates tácticos sobre un tablero cuadriculado donde los jugadores despliegan y gestionan unidades con jerarquías y características diferenciadas. Cada unidad posee fortalezas y debilidades específicas frente a otros tipos de unidades, lo que obliga a planificar los movimientos cuidadosamente y anticipar las acciones del adversario. Aunque la información sobre la ubicación de las unidades enemigas es visible, la profundidad estratégica recae en la gestión eficiente de recursos, el posicionamiento táctico y la secuencia de ataques, elementos que recuerdan la planificación y la jerarquía presentes en *Stratego*.

De manera similar, el videojuego *Into the Breach* (Subset Games, 2018) traslada el concepto de estrategia por turnos a un entorno de tablero discretizado donde cada unidad tiene capacidades específicas y limitadas. La mecánica principal consiste en anticipar el comportamiento del enemigo y planificar acciones para proteger objetivos clave, equilibrando riesgo y recompensa en cada turno. Esta combinación de toma de decisiones estrictamente calculada, jerarquía de unidades y confrontación directa recuerda la esencia de *Stratego*, donde el control del espacio y la gestión de las piezas son determinantes para la victoria. La simplicidad aparente del tablero contrasta con la profundidad estratégica requerida, consolidando a *Into the Breach* como un ejemplo contemporáneo de cómo los principios clásicos de los juegos de estrategia pueden adaptarse eficazmente al medio digital.

Estas aproximaciones digitales ponen de manifiesto la diversidad de soluciones adoptadas en el diseño de videojuegos de estrategia y sirven como referencia para el desarrollo de **NeuroForge**. El objetivo del proyecto es combinar la jerarquía de unidades y la confrontación directa propias de *Stratego* con las ventajas del medio digital, manteniendo la tensión estratégica derivada de la información oculta y aprovechando las posibilidades de automatización, accesibilidad y escalabilidad que ofrece el formato de videojuego.

3.4. Motor de videojuegos utilizado

Los motores de videojuegos funcionan como un *framework* de desarrollo, proporcionando un conjunto integrado de herramientas, librerías y sistemas que sirven como base tecnológica para la creación de un videojuego. Estos motores abstraen gran parte de la complejidad técnica inherente al desarrollo, ofreciendo soluciones ya implementadas para tareas como el renderizado gráfico, la gestión de escenas, el control de entrada del usuario, la simulación de físicas o la exportación a distintas plataformas. Gracias a esta abstracción, el desarrollador puede centrarse principalmente en la lógica del juego, el diseño de mecánicas y la experiencia del usuario.

La elección del motor de videojuegos influye de manera directa en el proceso de desarrollo y en las características finales del producto. Factores como el flujo de trabajo, el rendimiento, la escalabilidad, la facilidad de mantenimiento del código y los tiempos de implementación están fuertemente condicionados por las herramientas y la arquitectura que ofrece el motor seleccionado. En el contexto de un proyecto académico individual, esta decisión cobra especial relevancia, ya que deben considerarse además aspectos como la curva de aprendizaje, la disponibilidad de

documentación y recursos, y la adecuación del motor al alcance real del proyecto.

Con el objetivo de seleccionar el motor más adecuado para el desarrollo del videojuego *NeuroForge*, se realizó un análisis comparativo inicial de tres de los motores de desarrollo más utilizados en la actualidad: Unreal Engine, Unity y Godot. Estos motores cuentan con una amplia presencia tanto en el ámbito profesional como en el educativo y representan diferentes enfoques dentro del desarrollo de videojuegos [6].

Para realizar esta comparación se han considerado varios criterios relevantes para el contexto del proyecto. Entre ellos destacan el rendimiento del motor, la facilidad de aprendizaje, la adecuación al desarrollo de videojuegos en dos dimensiones, los requisitos de hardware, el ecosistema de herramientas disponibles y el modelo de licencias.

Unreal Engine es uno de los motores más utilizados dentro de la industria del videojuego, especialmente en producciones de gran presupuesto y alta exigencia gráfica. Sus avanzados sistemas de renderizado, iluminación y simulación física permiten desarrollar entornos tridimensionales altamente realistas, lo que lo convierte en una herramienta especialmente adecuada para proyectos de gran escala y alta fidelidad visual. Sin embargo, esta potencia técnica implica también una mayor complejidad en el flujo de trabajo y una curva de aprendizaje más pronunciada, especialmente para desarrolladores sin experiencia previa. Asimismo, los requerimientos de hardware suelen ser más elevados y los tiempos de compilación pueden resultar más largos en comparación con otros motores. Estas características pueden suponer una desventaja en proyectos académicos o desarrollos individuales donde el tiempo y los recursos disponibles son limitados.

Unity, por su parte, se ha consolidado como uno de los motores de videojuegos más populares tanto en el ámbito independiente como en el educativo. Su principal ventaja radica en su versatilidad, ya que permite desarrollar videojuegos en dos y tres dimensiones para una amplia variedad de plataformas. Además, cuenta con una comunidad muy extensa, una gran cantidad de documentación disponible y un ecosistema amplio de herramientas, extensiones y recursos que facilitan el aprendizaje y aceleran el desarrollo. No obstante, algunos análisis técnicos señalan que en proyectos con estructuras complejas o con altas exigencias gráficas pueden aparecer problemas de rendimiento si no se realiza una gestión cuidadosa de los recursos. Asimismo, aunque Unity ofrece soporte para el desarrollo bidimensional, su arquitectura interna está históricamente orientada al desarrollo tridimensional, lo que puede introducir cierta sobrecarga innecesaria en proyectos 2D más sencillos.

Finalmente, Godot es un motor de desarrollo de código abierto que ha ganado una notable popularidad en los últimos años, especialmente dentro del desarrollo independiente y el ámbito educativo. Su filosofía se basa en la simplicidad, la modularidad y la accesibilidad, lo que facilita su uso en proyectos de pequeña y mediana escala. Una de sus principales características es su arquitectura basada en nodos, que favorece un diseño modular y fácilmente mantenible del sistema. Además, Godot ofrece herramientas especialmente sólidas para el desarrollo de videojuegos en dos dimensiones, permitiendo ciclos de desarrollo rápidos y un prototipado ágil. A esto se suma la ausencia de costes de licencia y el acceso completo a su código fuente, lo que permite una mayor libertad de experimentación. Como principal limitación, el motor presenta capacidades menos maduras en el desarrollo de entornos tridimensionales complejos en comparación con motores

comerciales como Unreal Engine o Unity, así como un ecosistema de extensiones más reducido.

La Tabla 3.1 resume de forma sintética las principales características de los motores analizados, considerando los criterios más relevantes para el desarrollo del proyecto.

Criterio	Unreal Engine	Unity	Godot
Rendimiento en 3D	Muy alto	Alto	Medio
Rendimiento en 2D	Medio	Alto	Muy alto
Curva de aprendizaje	Alta	Media	Baja
Requisitos de hardware	Altos	Medios	Bajos
Modelo de licencia	Comercial	Comercial	Código abierto
Adecuación a proyectos individuales	Media	Alta	Muy alta
Ecosistema y comunidad	Muy amplio	Muy amplio	En crecimiento

Tabla 3.1: Comparativa entre motores de desarrollo de videojuegos

Tras el análisis realizado, se ha determinado que Godot es el motor más adecuado para el desarrollo del proyecto *NeuroForge*. Dado que se trata de un videojuego de estrategia por turnos en dos dimensiones, con un alcance controlado y desarrollado en un contexto académico individual, las ventajas de Godot en términos de simplicidad, rendimiento en 2D y ausencia de costes de licencia superan claramente sus limitaciones técnicas.

Además, su ligereza y rapidez en los ciclos de desarrollo favorecen un enfoque iterativo basado en pruebas frecuentes y refinamiento progresivo, lo que resulta especialmente adecuado para un proyecto desarrollado por una única persona y alineado con la metodología de trabajo adoptada en este TFG.



Figura 3.1: Tablero del Juego Real de Ur [1]



Figura 3.2: Tablero del juego Senet [2]

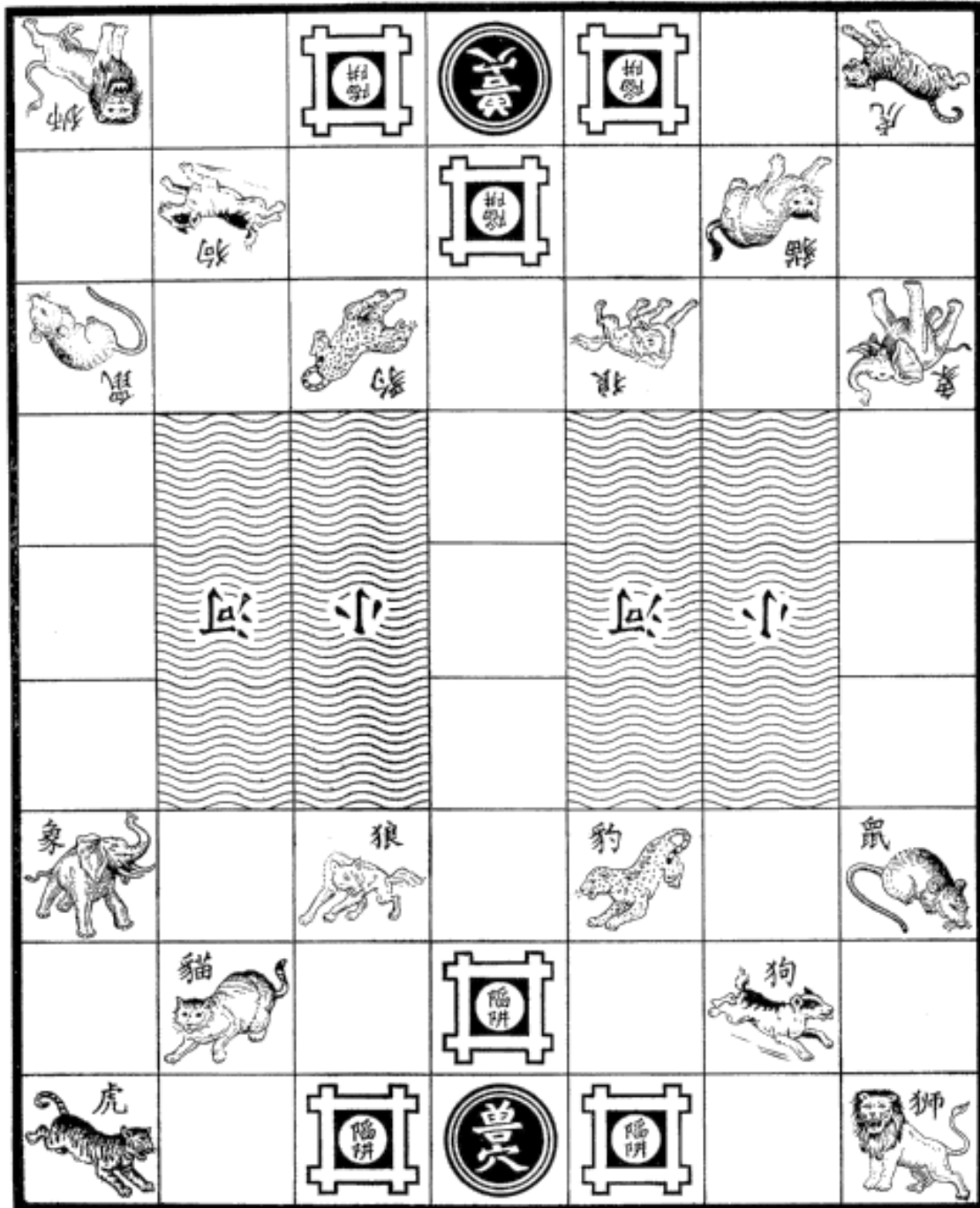


Figura 3.3: Tablero del juego Dou Shou Qi [3]



Figura 3.4: Tablero del juego L'attaque [4]

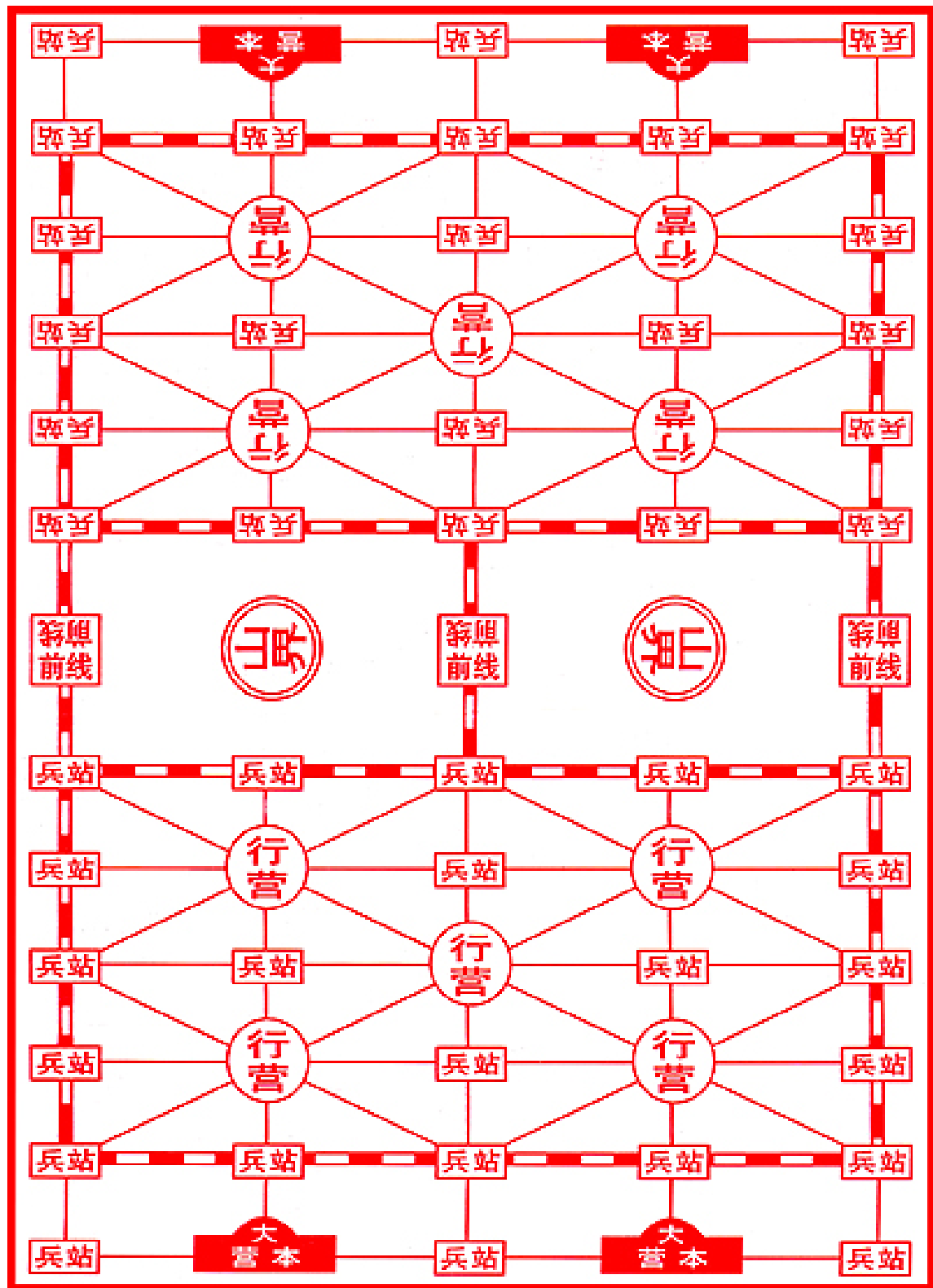


Figura 3.5: Tablero del juego Luzhanqi [5]

Concepto del videojuego

En este capítulo se describe el diseño general de **NeuroForge**, abordando sus mecánicas principales, reglas de juego y elementos que definen la experiencia estratégica. Se detallan tanto el funcionamiento básico del juego como las variaciones introducidas en sus distintos modos, junto con aspectos de interacción, ambientación y público objetivo.

4.1. Resumen del juego

NeuroForge es un videojuego de *estrategia por turnos* inspirado en el clásico *Stratego*. En él, dos oponentes se enfrentan en un tablero de 10×10 casillas con el objetivo de destruir el **núcleo de energía** rival o dejarlo sin movimientos posibles.

Cada oponente recibe un conjunto de piezas de distintos rangos, que coloca en secreto dentro de su zona de despliegue (las 4 filas más cercanas a su borde del tablero). Las piezas permanecen ocultas para el oponente hasta que entren en combate.

En su turno, cada oponente puede realizar una de 2 acciones

- **Moverse**, que consiste en desplazar una pieza a una de sus 4 casillas ortogonales adyacentes, salvo algunas que no puedan moverse o un tipo de pieza en especial, que se pueden mover varias casillas en línea recta.
- **Atacar**, cuando una pieza se mueve a una casilla ocupada por un enemigo, ambas revelan su identidad y se resuelve el combate según su rango.

Por lo general el combate de las piezas se resuelve de la siguiente forma:

- La pieza de mayor rango vence a la de menor rango, y ocupa la casilla enemiga o conserva la suya en caso de que la atacante sea la perdedora.
- Si ambas piezas tienen el mismo rango, ambas pierden y se eliminan del tablero.

Luego existen algunas piezas que tienen habilidades o interacciones especiales que se explicaran más adelante, como que la del rango mas bajo puede eliminar a la del mas alto si ataca primero.

El juego finaliza cuando o bien, un jugador destruye el **núcleo de energía** del oponente, o uno de los oponentes no tiene la capacidad de realizar mas movimientos.

4.2. Mecánicas

NeuroForge en principio contara con un oponente humano, el jugador, y el otro sera la propia maquina, un bot que controlara el bando enemigo.

Gran parte del atractivo de *NeuroForge* reside en el desconocimiento de la identidad de las piezas de ambos oponentes, lo que resulta en la deducción y el engaño como parte de la estrategia. Sin embargo, este mismo factor puede generar situaciones frustrantes o en algunos casos depender excesivamente del azar, o incluso llegar generar situaciones de pasividad en las que ambos oponentes se quedan a la espera de que el otro ataque primero. Por ello, se han definido dos modos de juego distintos:

- **Clásico:** Modo basado en la experiencia del juego base, con mecánicas sencillas y un fuerte componente estratégico en el que la deducción es el factor principal.
- **Moderno:** Incluye más piezas y reglas adicionales, con el objetivo de disminuir el peso de la deducción y ofrecer partidas más claras y menos ambiguas respecto a la identidad de las piezas sin excederse.

A continuación se explicará en profundidad el contenido del juego y todas sus mecánicas, y las diferencias o añadidos que tiene el modo **Moderno** frente al **Clásico**

4.2.1. Tablero

NeuroForge inicialmente cuenta con un tablero cuya estructura es una cuadrícula de 10×10 casillas (Figura 4.1). Cada oponente dispone de las 4 filas más cercanas a su lado (10×4) como área de despliegue, para la colocación inicial de sus piezas sin que el rival conozca ninguna de sus identidades. Una vez comenzada la partida ya se pueden mover libremente.

También existen **casillas intransitables**, es decir, que ninguna pieza bajo ningún concepto puede moverse a ellas o saltarlas. En este tablero se encuentran en el centro, formando 2 zonas de dimensiones 2×2 .

4.2.2. Colocación inicial

Al inicio de una partida, cada oponente colocará sus piezas dentro de su respectiva área de despliegue con la distribución que quieran siempre. No se le permitirá colocar ninguna pieza

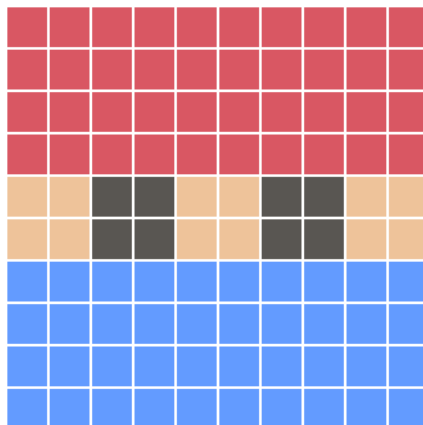


Figura 4.1: Esquema tablero

fuera de dicho área antes de comenzar la partida, las 2 filas centrales del tablero permanecerán despejadas al inicio de la partida.

Una vez la colocación de las piezas esté decidida, se podrá comenzar la partida. El jugador no conocerá la identidad de ninguna de las piezas rivales, y el bot tampoco sabrá cuales son las del jugador.

4.2.3. Piezas

Principalmente las piezas se rigen por un sistema de rangos a la hora de combatir, donde las de mayor rango ganan a las de menor rango, y hay una cantidad limitada por cada tipo de pieza. Hay piezas que tienen propiedades adicionales, ya sea en combate o en su movimiento. Los tipos de piezas y sus propiedades son los siguientes:

- **Núcleo de energía (cantidad 1):** Es la pieza mas importante, su destrucción implica la derrota inmediata del propietario. No puede moverse y puede ser destruido por cualquier pieza enemiga que lo ataque.
- **Torretas automáticas (cantidad 6):** Permanecen inmóviles, no tienen la capacidad de atacar pero destruyen cualquier pieza que las ataque independientemente de su rango, salvo los **saboteadores** que son los únicos capaces de destruirlas.
- **C.O.R.E. (cantidad 1, rango 10):** Si es atacado por el **Phantom** pierde.
- **Guardia Nova (cantidad 1, rango 9)**
- **Mecha (cantidad 2, rango 8)**
- **Androide de élite (cantidad 3, rango 7)**
- **Unidad de combate (cantidad 4, rango 6)**
- **Armero (cantidad 4, rango 5)**
- **Soldado (cantidad 4, rango 4)**

- **Saboteadores (cantidad 5, rango 3):** Es la única pieza capaz de destruir las **torretas automáticas**.
- **Exploradores (cantidad 6, rango 2):** Son capaces de desplazarse cualquier número de casillas en línea recta tanto horizontal como verticalmente siempre y cuando no haya obstáculos u otra pieza en medio que le impidan su paso. En caso de que se muevan mas de una casilla en un solo movimiento, **su identidad sera revelada**.
- **Phantom (cantidad 1, rango 1):** Es capaz de derrotar al **C.O.R.E.** solo si **ataca primero**.

Piezas especiales

En el caso del modo **Moderno** se añadirían 2 piezas nuevas:

- **Torre de radar (1 por jugador):** Esta pieza dispone de un máximo de **5 cargas**. Cada vez que su dueño participe en un combate (independientemente del resultado), la torre gana 1 carga. Cuando alcanza las 5 cargas, la torre queda **lista para activarse**.

El jugador puede entonces usar la torre para **revelar la identidad** de una pieza enemiga de su elección que aún no haya sido descubierta. Tras su activación, la torre se vacía y vuelve a 0 cargas, por lo que deberá recargarse de nuevo para poder usarse otra vez. Si la torre ya está cargada al máximo y se producen nuevos combates sin haberla activado, esas cargas se **pierden**.

- **Inhibidor (cantidad 1):** Genera un campo de interferencia en su propia casilla y en las ocho adyacentes. Mientras permanezca en el tablero, las piezas de su campo no pueden ser reveladas por el radar enemigo. Se introduce como un contrapeso a la ventaja que supone la **torre de radar**.

Ambas piezas, al igual que el núcleo, no pueden moverse y pueden ser destruidas por cualquier pieza enemiga que las ataquen.

El papel de estas piezas, es el de **desincentivar la pasividad**, se premia al oponente que inicia los ataques, con una herramienta para poder revelar la identidad de una pieza a elección, dándole una ventaja sobre el rival en caso de jugar mas agresivo. Pero que no merezca la pena, en caso de que haga ataques sin pensar, ya que el escaneo se puede ver frustrado por el **Inhibidor**.

4.2.4. Turnos

La partida se desarrolla por turnos alternos, en el que el jugador empieza primero. En un turno pueden hacerse 2 cosas, **mover** o **atacar** con una pieza.

No se puede moverse y atacar en el mismo turno, a excepción de los **exploradores** al poder moverse varias casillas. Si ninguna de las piezas de un bando son capaces de moverse o de atacar, la partida se acaba y dicho bando pierde.

Reglas para moverse

- Solo se puede mover una pieza por turno, y solo una casilla (excepto el **explorador**). Pueden moverse en cualquiera de las 4 direcciones adyacentes (al frente, atrás, izquierda y derecha), pero nunca en diagonal.
- No puede haber más de una pieza ocupando la misma casilla al mismo tiempo.
- Las piezas no pueden saltar o atravesar una casilla ocupada por otra pieza.
- Las piezas no podrán moverse ni estar en casillas intransitables, tampoco se podrán saltar o atravesar por ellas.
- Una vez desplazada la pieza (explicado mejor en la sección de interacción y control), el turno habrá finalizado y será turno del oponente.
- Una pieza no puede desplazarse de manera repetitiva entre las mismas casillas. En concreto, una vez que una pieza abandona una casilla, deberá esperar al menos **tres turnos completos** antes de poder volver a ocuparla. Este límite solo afecta a la pieza que salió de la casilla: otras piezas sí pueden entrar en ella sin restricción.
- La pieza tiene que tener la capacidad de poder moverse, por ejemplo el **núcleo de energía** y las **torretas automáticas** no pueden moverse, su posición es permanente para el resto de la partida.

Reglas para atacar

- Si te mueves a una casilla ocupada por una pieza del oponente, cuenta como atacar. Si la pieza no puede moverse a esa casilla, no se puede atacar.
- Una vez se ha decidido atacar a una pieza rival se entra en combate (mas adelante se explica la resolución del mismo).
- Si la pieza atacante gana el combate, pasa a ocupar la casilla rival y esta es destruida. Si pierde el combate, esta se destruye, y la pieza rival a la que ataco, permanece en su sitio.

En caso de estar jugando en el **modo moderno**, la **torre de radar** no puede moverse, por ende no puede atacar, pero si uno de los oponentes puede activarla, se podrá seleccionar dicha pieza y acto seguido poder seleccionar una pieza rival sin identificar, para revelarla. Esta acción contara como ataque y **consumirá el turno** de quien la use.

4.2.5. Resolución de combates

Como bien se ha dicho, cuando una pieza se mueve una casilla ya ocupada por una pieza rival, se produce un combate. Cuando se produce un combate, la identidad de ambas piezas se revelan, y el resultado está determinado por una serie de reglas:

- Si la pieza atacante tiene **mayor rango** que la pieza defendida, **vence** el combate: la pieza rival es destruida y el atacante ocupa su casilla.
- Si la pieza atacante tiene **menor rango**, **pierde** automáticamente: es destruida y la pieza defendida permanece en su posición.
- Si ambas piezas tienen el **mismo rango**, el combate termina en **empate** y ambas son destruidas.
- Una **torreta automática** destruye a cualquier pieza que la ataque, excepto a los **exploradores**.
- Si un **explorador** ataca a una **torreta automática**, la destruye y ocupa su casilla.
- Si un **Phantom** ataca a un **C.O.R.E.**, el **Phantom vence**, eliminando al C.O.R.E.

Reglas adicionales para el **modo moderno**:

- La **torre de radar** solo podrá revelar una unidad rival que no este identificada.
- La **torre de radar** solo podrá revelar si esta cargada al máximo, de lo contrario no podrá activarse.
- La **torre de radar** y el **inhibidor** son destruidas al recibir el ataque de cualquier pieza enemiga.

Adicionalmente en el **modo moderno**, antes de iniciar la partida, se podrán decidir que reglas adicionales de las siguientes poder activarse o no para una partida:

- **Refuerzos:** Si una pieza alcanza la última fila del tablero, puede **solicitar refuerzos** de una pieza eliminada en combate, devolviendo al tablero la pieza donde **el solicitante elija** dentro de su zona de despliegue. Si se da el caso de no haber una libre para ello no se podrá solicitar refuerzos.

El rival recibe únicamente la notificación de que se su oponente ha solicitado un refuerzo, pero no cual.

La acción de solicitar refuerzos consumirá el turno del propietario, por lo que ya tiene que haber una pieza suya a su otro extremo del tablero para poder hacerlo.

Existen las siguientes restricciones:

- Solo se pueden solicitar **piezas con rango** que hayan sido eliminadas, es decir **torretas automáticas**, **torre de radar** y el **inhibidor** no pueden ser solicitados.
 - Los **exploradores** no pueden solicitar refuerzos.
 - Cada jugador dispone de un **máximo de dos solicitudes** de refuerzo.
 - Una misma pieza **no puede solicitar** más de un refuerzo.
- **Ventaja de Ataque:** En el caso de que en un combate, ambas piezas enfrentadas tengan de mismo rango, ganara la pieza que inicio el combate.

4.2.6. Condiciones de victoria

Existen solo 2 formas de ganar:

- El núcleo de energía rival es destruido.
- El rival se ha quedado sin movimientos posibles que hacer, todas sus piezas con capacidad de moverse han sido destruidas.

4.2.7. Interacción y control

El usuario desde principio a fin de la partida podrá realizar todas las acciones que necesite con el ratón. El flujo de acciones está diseñado para que todas las posibilidades sean intuitivas y visualmente claras, de la siguiente forma:

- **Turno del jugador:** al comenzar su turno, el jugador solo ve resaltadas las piezas que pueden moverse, mientras que el resto que no puedan moverse no destaquen.
- **Acción:** Para mover una pieza resaltada, el jugador debe mantener pulsado el botón del ratón sobre ella. Mientras la mantenga seleccionada, se mostrarán las casillas disponibles para su desplazamiento. El jugador deberá arrastrar la pieza hasta la casilla resaltada deseada y, al soltar el ratón, el movimiento queda confirmado. El juego no le permitirá mover la pieza a una casilla que no pueda (solo las resaltadas). Si la pieza se suelta en su casilla de origen, la acción no se considera válida y el jugador podrá volver a mover hasta realizar un movimiento válido. Esto último es la única forma de cancelar un movimiento en caso de arrepentirse antes de llevarlo a cabo. Al realizar un movimiento válido sucederá una de las siguientes posibilidades:
 - **Movimiento:** Si la pieza se desplaza a una casilla vacía no sucede nada más y se pasa el turno.
 - **Combate:** Al desplazarse a una casilla ocupada por una pieza enemiga, ambas piezas entrarán en combate. Mediante una breve animación se resaltan ambas piezas en pantalla. Si la pieza enemiga estaba oculta, se revela con un efecto adicional. Se decide la pieza vencedora de acuerdo a las reglas de combate, y la pieza perdedora se destruye y desaparece del tablero. Posteriormente se verá la pieza vencedora en la casilla en la que se originó el combate.
- **Retroalimentación audiovisual:** cada acción se acompaña de animaciones breves y efectos sonoros.
- **Turno del bot:** una vez concluida la acción del jugador, se inicia el turno del oponente. El jugador observa su jugada y espera hasta recuperar nuevamente el control.

Para las reglas adicionales del **modo moderno**:

- **Radar:** La pieza no se resalta en el tablero, pero su estado puede consultarse en la interfaz, donde se muestra el número de cargas acumuladas y un botón de activación. Cuando la torre alcanza todas sus cargas, el botón se habilita y el jugador puede decidir activarla. Una vez activada, el botón cambia para mostrar la posibilidad la opción de cancelar la habilidad en caso de retractarse. También, se resaltan las piezas enemigas ocultas como posibles objetivos.

Al seleccionar una de ellas, una animación revela su identidad. Si el escaneo es bloqueado por un **inhibidor**, se notificará al jugador mediante un aviso visual y sonoro.

Tras el uso de la habilidad, todas las cargas se consumen y el botón de activación permanece bloqueado hasta que la torre vuelva a recargarse por completo.

- **Refuerzos:** Si un jugador consigue situar una de sus piezas en el extremo opuesto del tablero, la interfaz mostrará una opción adicional para ejecutar la acción de **refuerzo**. Al igual que con la habilidad del radar, esta acción puede confirmarse o cancelarse en cualquier momento.

Al activarla, el jugador debe elegir qué pieza solicitar como refuerzo. Una vez seleccionada, se abre un menú con las piezas eliminadas disponibles para ser recuperadas como refuerzo. En este punto, es posible volver atrás para elegir otra pieza para solicitar o cancelar la acción.

Tras escoger la unidad a solicitar, el jugador selecciona una casilla libre dentro de su zona de despliegue para colocarla. Si todas las casillas de la zona están ocupadas, no será posible completar la solicitud de refuerzo, y al jugador no le quedará mas remedio que cancelar el rescate. El juego informa al jugador del motivo.

4.3. Ambientación

La narrativa de *NeuroForge* sitúa al jugador en un universo de **ciencia ficción futurista**, donde la guerra se libra en entornos dominados por inteligencias artificiales, drones y armamento avanzado. El propio título del juego, *NeuroForge*, hace referencia al proceso de **forjar una red neuronal**, cada partida representa un ciclo de entrenamiento intensivo en el que el jugador moldea y perfecciona una IA militar para prepararla frente a futuros conflictos.

De este modo, el enfrentamiento entre bandos no es una batalla real, sino parte de un **programa avanzado de simulación**, diseñado para medir, reforzar y optimizar las capacidades estratégicas de estas inteligencias. Cada combate, victoria o derrota contribuye al forjado de la red neuronal de esa IA, preparándola cada vez mas para distintos escenarios.

Descripción de piezas

A continuación, se presenta una breve descripción de las distintas unidades y estructuras que intervienen en el juego:

- **Núcleo de energía:** Centro vital de cada bando, representa la fuente principal de energía

que mantiene a las unidades en pie. Su destrucción implica la pérdida de suministro de energía y por tanto la inminente derrota del adversario.

- **Torretas automáticas:** Sistemas defensivos estáticos con la potencia capaz de eliminar a cualquier amenaza, salvo los **saboteadores** que las eluden.
- **C.O.R.E. (Command Operations and Recon Elite):** Robot mecha instruida personalmente por la inteligencia artificial de máximo nivel a cargo de su bando, la unidad más poderosa del campo de batalla.
- **Guardia Nova:** Líder cyborg con partes orgánicas casi inexistentes de alto rango que coordina el despliegue de las fuerzas en el campo de batalla.
- **Mecha:** Robots pesados a menor escala que el del **C.O.R.E.** diseñados para resistir y dominar en combate directo.
- **Androide de élite:** Androides hábiles y optimizados para dominar en la mayoría de combates a corta distancia.
- **Unidad de combate:** Androides con un amplio arsenal para valerse en batalla y con la capacidad de mando sobre fuerzas menores.
- **Armero:** Soldados rápidos y eficaces con ligeros implantes cibernéticos.
- **Soldado:** Refuerzos para mantener la líneas de combate completamente humanos.
- **Saboteador:** Soldados especializados capaces de disuadir y desactivar torretas automáticas mediante camuflaje de alta tecnología, pero indefensos en combate.
- **Explorador:** Drones de reconocimiento bastante ágiles que pueden recorrer largas distancias para explorar y obtener información. No van armados mas allá de sus herramientas para repararse de daños ambientales.
- **Phantom:** Dron ágil diseñado para infiltrarse y destruir a la IA de máximo nivel con el uso de malware, siempre que ataque primero. No son muy eficientes debido a que el malware que portan capaz de colapsar un modelo de **C.O.R.E.**, los ciegan a cumplir ese único propósito, haciéndolos fáciles de destruir por otras unidades en combate.

Escenario

Como ya se mencionó, en el mapa existen zonas intransitables, las cuales consisten en cúmulos de **campos de plasma inestable**. Se trata de restos de antiguas batallas, cementerios energéticos formados por residuos de armamento avanzado.

Estas áreas generan descargas aleatorias e intensas interferencias que no solo las convierten en territorios letales, sino que además **imposibilitan el funcionamiento** de cualquier unidad dependiente de tecnología o con capacidad de sobrevolarlas. Su letalidad unicamente se puede reducir con el paso del tiempo en caso de no ser contenido adecuadamente.

4.4. Diseño de niveles

NeuroForge parte de una base sencilla: un **tablero de 10×10 casillas**. Pese a su aparente simplicidad, este escenario ofrece una amplia profundidad táctica gracias a la presencia de **zonas intransitables** en el centro, que actúan como cuellos de botella. Estos obstáculos dividen el tablero en dos mitades y obligan a los jugadores a planificar cuidadosamente sus rutas de ataque, defensa y exploración, condicionando las aperturas iniciales y las estrategias de movimiento.

Actualmente, el diseño se centra exclusivamente en este tablero estándar. Sin embargo, el planteamiento del juego abre la puerta a una serie de posibilidades de expansión futura:

- **Mapas irregulares:** Tableros con dimensiones distintas al estándar (por ejemplo, 12×12 o rectangulares 8×14), que modifiquen la densidad de las tropas y el tiempo de las partidas. También que el tablero no sea simétrico o bordes imperfectos.
- **Eventos ambientales:** Incorporación de fenómenos que alteren temporalmente el tablero (tormentas de plasma, desprendimientos...) y afecten a la movilidad o visibilidad de ciertas piezas temporal o permanentemente durante el resto de la partida.
- **Zonas estratégicas:** Casillas especiales que otorguen ventajas al ser controladas (por ejemplo, teletransportes, potenciador de rango para la pieza que la obtenga, recuperación de piezas eliminadas o cargas adicionales para el radar).

De esta forma, aunque el juego base se centra en un único tablero, su diseño modular, permite imaginar un ecosistema y crear niveles con facilidad, permitiendo tener un **alto nivel de rejugabilidad**, sin llegar a alterar las mecánicas fundamentales del combate por turnos.

4.5. Audiencia - Población objetivo

El videojuego está orientado a jugadores interesados en la estrategia por turnos, tanto aquellos con experiencia previa en el género como quienes quieran adentrarse en él por primera vez. El público objetivo incluye a personas con afinidad por juegos de lógica, planificación y táctica, como el ajedrez o títulos clásicos de estrategia, independientemente de su edad.

Aunque por su estilo visual y sus reglas puede resultar especialmente atractivo para un público **joven y adulto**, el diseño busca ser accesible y comprensible para cualquier jugador con interés en los retos estratégicos, priorizando la claridad en las mecánicas y la profundidad en la toma de decisiones.

Desde el punto de vista de la clasificación por edades, el contenido del juego no incluye violencia explícita, lenguaje ofensivo ni temáticas sensibles. Atendiendo a los criterios del sistema de clasificación europeo *Pan European Game Information* (PEGI), es razonable considerar que el videojuego podría situarse dentro de una clasificación aproximada de **PEGI 3** o **PEGI 7**, al tratarse de una experiencia estratégica abstracta sin representaciones realistas de violencia.

Planificación

En este capítulo se presenta la planificación del desarrollo del proyecto *NeuroForge*, definiendo la organización temporal del trabajo, los recursos empleados, las restricciones existentes y los posibles riesgos identificados.

La planificación se ha concebido como una guía flexible, susceptible de ser ajustada a lo largo del desarrollo conforme se adquiría una visión más precisa del alcance real del proyecto y de las dificultades técnicas asociadas a su implementación.

5.1. Riesgos

Durante la fase de planificación se identificaron distintos riesgos que podrían afectar al correcto desarrollo del proyecto, los cuales estan recogidos en la Tabla [5.1](#). Para cada uno de ellos se ha analizado su naturaleza y su impacto potencial, definiendo también medidas de contingencia orientadas a minimizar sus efectos en caso de que llegaran a ocurrir.

La identificación temprana de estos riesgos permite anticipar posibles problemas técnicos y organizativos, facilitando la toma de decisiones durante el desarrollo del proyecto y contribuyendo a mantener la estabilidad del mismo.

Riesgo	Tipo	Impacto	Plan de contingencia
R1: Fallos del motor o incompatibilidades	Técnico	Alto	Revisar dependencias o adaptar scripts.
R2: Retrasos en el cronograma	Temporal	Medio	Reajustar tareas no prioritarias y reducir funcionalidades accesorias.
R3: Errores en el Bot o en las reglas del juego	Técnico	Medio	Simplificar el comportamiento de la IA y priorizar mecánicas estables.
R4: Pérdida de datos o código fuente	Técnico	Alto	Uso de control de versiones y copias de seguridad en la nube.
R5: Falta de tiempo para testeo o balanceo	Temporal	Alto	Priorizar pruebas de jugabilidad y corregir errores críticos.
R6: Carencia de recursos gráficos o sonoros	Logístico	Bajo	Crear recursos propios o editar recursos de terceros de libre uso.
R7: Incapacidad temporal de desarrollo	Temporal	Alto	Reorganizar tareas, priorizar funcionalidades críticas y ajustar cronograma.
R8: Dificultad en aprendizaje o implementación del algoritmo del bot	Técnico	Medio-Alto	Simplificar temporalmente el bot a reglas básicas; mantener versión mínima funcional.
R9: Complejidad inesperada en reglas modernas o habilidades especiales	Técnico	Bajo	Implementar primero el modo clásico; introducir gradualmente las reglas modernas; validar cada mecánica individualmente.
R10: Bloqueos por depuración de bugs difíciles	Técnico	Medio	Mantener control de versiones; realizar pruebas unitarias frecuentes; documentar problemas; revisar documentación oficial de Godot y C#.

Tabla 5.1: Riesgos identificados y planes de contingencia actualizados

5.2. Restricciones

El desarrollo de *NeuroForge* ha estado condicionado por una serie de restricciones que han influido directamente en la planificación del proyecto, en la selección de herramientas y en el alcance final de las funcionalidades implementadas. Siendo las principales restricciones:

- **Técnicas:** Se establece el uso del motor de videojuegos *Godot* junto con el lenguaje de programación *C#*. Aunque *Godot* emplea de forma nativa *GDScript*, se ha optado por *C#* debido a su soporte oficial y a su adecuación al desarrollo del proyecto. Asimismo, únicamente se utilizan librerías y recursos gratuitos o de código abierto.
- **Temporales:** El desarrollo del proyecto se limita al periodo de realización del Trabajo de Fin de Grado, lo que impone una duración finita y condiciona la cantidad de funcionalidades que pueden implementarse.
- **Económicas:** No se dispone de presupuesto para la adquisición de licencias de software de pago, la contratación de personal adicional ni la compra de recursos gráficos o sonoros.
- **Personales:** El proyecto es desarrollado íntegramente por un único estudiante, lo que limita la paralelización de tareas y requiere una gestión eficiente del tiempo disponible.
- **Académicas:** El trabajo debe ajustarse a la normativa, estructura y requisitos establecidos por la Escuela de Ingeniería Informática de Valladolid para los Trabajos de Fin de Grado.

5.3. Herramientas

Para el desarrollo del TFG se han empleado distintas herramientas de software tanto para la implementación del videojuego como para la gestión del proyecto y la redacción de la memoria. La selección de estas herramientas se ha realizado atendiendo a criterios de adecuación técnica, disponibilidad gratuita y facilidad de uso, buscando en todo momento optimizar el proceso de desarrollo. En la Tabla 5.2 se recoge un resumen de las principales herramientas utilizadas y su función dentro del proyecto.

El desarrollo del videojuego se ha realizado utilizando el motor de videojuegos *Godot*, un entorno de desarrollo multiplataforma de código abierto que proporciona un conjunto completo de herramientas para la creación de videojuegos, incluyendo sistemas de escena, gestión de nodos, físicas, interfaz gráfica y soporte para distintos lenguajes de programación. En este proyecto se ha utilizado *Godot* como base para la implementación de la lógica del juego, la gestión del tablero, las interacciones del jugador y la representación visual del estado de la partida.

Para la programación de la lógica del juego se ha utilizado el lenguaje *C#*, que cuenta con soporte oficial dentro del motor *Godot*. La elección de este lenguaje se debe a su robustez, su tipado estático y su amplia adopción en el desarrollo de software, lo que facilita la organización del código y el mantenimiento del proyecto a medida que aumenta su complejidad.

Durante el desarrollo también se ha empleado un sistema de control de versiones basado en *Git*, utilizando la plataforma *GitHub* como repositorio remoto. Esta herramienta ha permitido llevar un seguimiento de los cambios realizados en el código, mantener un historial de versiones del proyecto y disponer de una copia de seguridad del mismo en la nube.

En cuanto a la creación y edición de recursos visuales, se han utilizado herramientas de diseño gráfico como *Aseprite* y *Paint*. Estas aplicaciones han servido para generar o modificar

sprites y otros elementos visuales necesarios para representar el tablero, las piezas y los distintos componentes de la interfaz del juego.

Para la gestión de recursos sonoros se han empleado herramientas como Audacity, junto con recursos procedentes de la plataforma Freesound. Audacity ha permitido realizar pequeñas ediciones o ajustes sobre efectos de sonido, mientras que Freesound se ha utilizado como fuente de recursos de audio de libre uso que pueden incorporarse al proyecto respetando sus licencias.

La redacción y maquetación de la memoria del TFG se ha realizado mediante $\text{\LaTeX}$, un sistema de preparación de documentos ampliamente utilizado en entornos académicos y científicos que facilita la estructuración del contenido, la generación automática de referencias y la gestión de figuras y tablas.

Finalmente, para la planificación y organización de las tareas del proyecto se ha utilizado Notion, una herramienta de gestión que ha permitido estructurar las distintas fases del desarrollo, definir hitos y realizar un seguimiento del progreso del trabajo a lo largo del tiempo.

Tipo	Herramienta	Uso principal
Motor de juego	Godot	Desarrollo del videojuego, lógica, interfaz...
Lenguaje de programación	C#	Programación de la lógica de juego y scripts.
Control de versiones	Git / GitHub	Gestión de versiones, control de cambios y copia de seguridad del código.
Diseño gráfico	Aseprite / Paint	Creación y/o edición de sprites, texturas y elementos visuales en caso de necesitarse.
Audio	Audacity / Freesound	Creación y/o de efectos sonoros y música de fondo en caso de necesitarse.
Documentación	$\text{\LaTeX}$	Redacción y maquetación del presente documento del TFG.
Gestión de tareas	Notion	Planificación de fases, control de hitos y organización del flujo de trabajo.

Tabla 5.2: Herramientas

5.4. Planificación inicial

La planificación inicial del proyecto se realizó con el objetivo de establecer una estimación razonable del trabajo a desarrollar, sirviendo como referencia para organizar las tareas y distribuir el esfuerzo a lo largo del periodo de realización del TFG. Esta planificación no se concibe como un plan rígido, sino como una guía orientativa, susceptible de ajustes conforme avanzaba el desarrollo del proyecto.

El desarrollo del videojuego se planteó siguiendo una metodología iterativa, de modo que cada fase se centra en un conjunto concreto de objetivos y funcionalidades, permitiendo validar progresivamente el estado del proyecto y añadir nuevas capacidades sobre una base previamente funcional. La primera iteración, que se estima en unas 20 horas de trabajo, se dedicó a implementar

las mecánicas fundamentales del juego, incluyendo la estructura del tablero, la definición de las piezas, el sistema de movimiento, el combate entre piezas, la gestión del estado del tablero y el sistema de turnos. El objetivo principal de esta fase era obtener una primera versión completamente jugable del juego.

La segunda iteración, con una duración aproximada de 40 horas, estuvo centrada en el desarrollo del bot, un oponente controlado por inteligencia artificial capaz de identificar movimientos válidos y participar en la partida respetando las reglas del sistema. Inicialmente, se planteó un comportamiento básico basado en la selección de movimientos válidos, dejando abierta la posibilidad de mejorar el algoritmo mediante técnicas de aprendizaje o refinamiento posterior.

Durante la tercera iteración, que se prolongó durante unas 60 horas, se trabajó en el diseño y desarrollo de la interfaz de usuario del videojuego. Esta fase incluyó la representación visual del tablero y de las piezas, la incorporación de distintos indicadores de estado de la partida, así como el bocetado inicial de la interfaz y la creación de menús básicos de interacción con el jugador.

Finalmente, la cuarta iteración, con una estimación de 40 horas, se centró en las mejoras audiovisuales y el pulido final del proyecto. Durante esta etapa se incorporaron animaciones, efectos sonoros y mejoras visuales, al tiempo que se realizaron tareas de optimización del rendimiento del sistema y corrección de errores detectados durante las pruebas.

Paralelamente al desarrollo técnico del videojuego, se dedicaron aproximadamente 140 horas a la documentación del proyecto. Esta labor consistió en la redacción progresiva de la memoria del Trabajo de Fin de Grado, incluyendo la descripción del diseño del videojuego, la justificación de las decisiones técnicas adoptadas y el análisis de los resultados obtenidos durante el desarrollo.

En conjunto, la estimación total de dedicación para el proyecto asciende a unas 300 horas, de las cuales 160 se destinaron al desarrollo del videojuego distribuido entre las distintas iteraciones, mientras que las 140 restantes se dedicaron a la elaboración progresiva de la documentación, realizada de forma paralela con el desarrollo del sistema.

La Tabla 5.3 recoge de forma resumida las iteraciones planificadas y la fecha estimada de finalización de cada una de ellas dentro del cronograma del proyecto.

Iteración	Descripción	Fecha estimada de finalización
1	Desarrollo del juego base	2026-03-15
2	Implementación del bot	2026-04-12
3	Desarrollo de la interfaz de usuario	2026-05-17
4	Mejoras audiovisuales y pulido final	2026-05-31
Documentación	Redacción de la memoria del TFG	2026-06-7

Tabla 5.3: Fechas estimadas de finalización de las iteraciones del proyecto

5.5. Cronograma del proyecto

La planificación temporal del desarrollo de *NeuroForge* se ha representado mediante un diagrama de Gantt que permite visualizar de forma clara todas las fases del proyecto, su duración estimada y la secuencia de tareas realizadas durante el desarrollo.

La Figura [5.1](#) muestra de manera integral todas las iteraciones del proyecto junto con la tarea de documentación, que se realiza de forma continua desde el inicio hasta la finalización del trabajo.

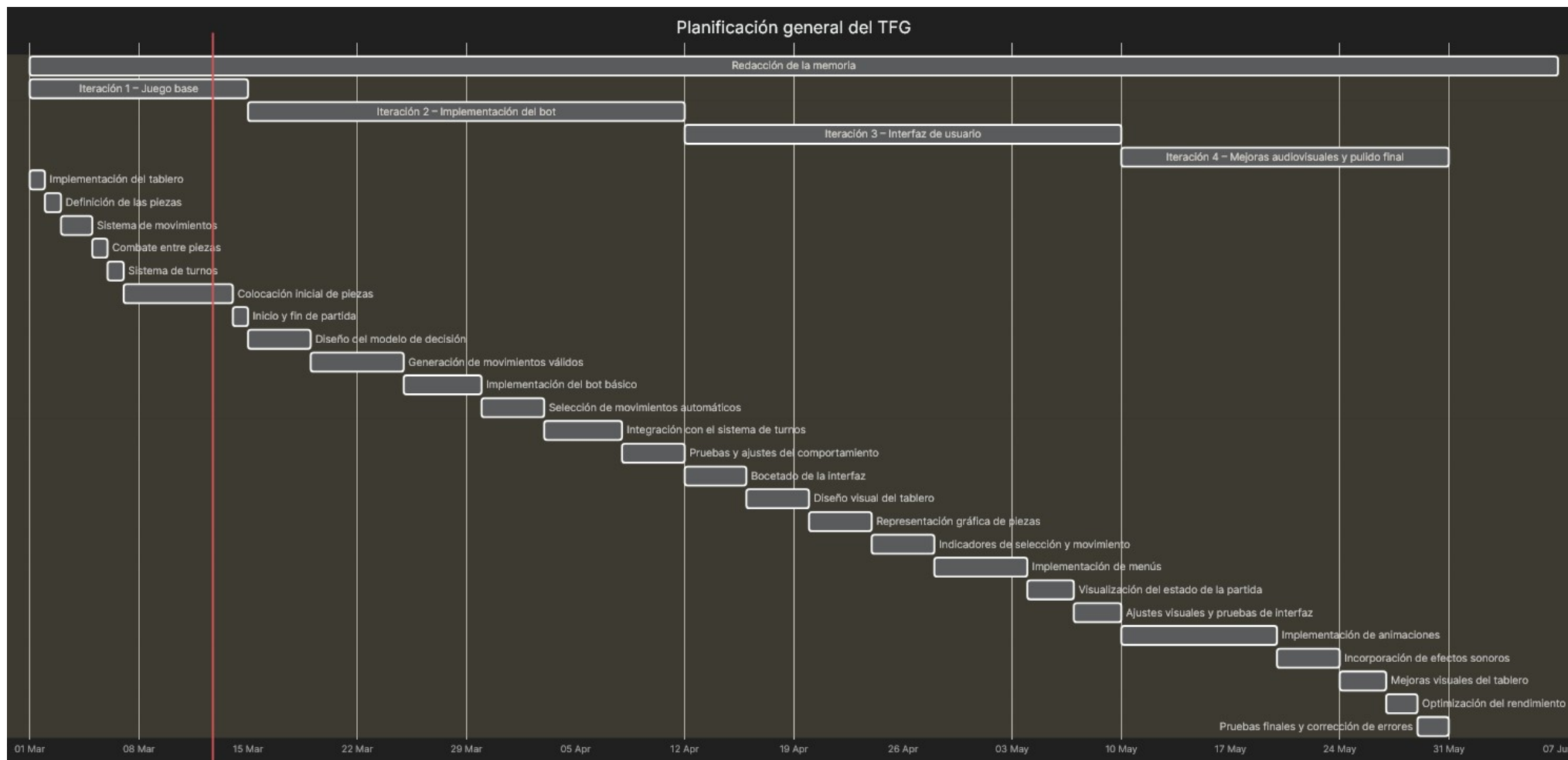


Figura 5.1: Diagrama de Gantt general del desarrollo del TFG, incluyendo todas las iteraciones y la documentación

5.6. Variación de planificación

...

5.7. Costes del proyecto

El desarrollo del Trabajo de Fin de Grado se ha realizado en un contexto académico, por lo que no se han producido costes económicos directos asociados al proyecto más allá de la matrícula correspondiente. Todas las herramientas empleadas durante el desarrollo han sido de uso gratuito, de código abierto o bien se han utilizado mediante licencias académicas proporcionadas por la universidad.

No obstante, desde un punto de vista hipotético, si este proyecto se hubiera desarrollado en un entorno profesional, el principal coste asociado correspondería al tiempo de trabajo invertido en su desarrollo.

La dedicación total estimada para el desarrollo del proyecto ha sido de aproximadamente 300 horas, distribuidas entre las distintas fases de análisis, diseño, implementación, pruebas y documentación. Considerando un salario mensual aproximado de 1.800 euros brutos para un desarrollador junior y una jornada laboral media de 160 horas mensuales, el coste horario estimado sería de aproximadamente 11,25 euros por hora.

Bajo estas condiciones, el coste asociado al tiempo de desarrollo sería:

- Coste de desarrollo: $300 \text{ h} \times 11,25 \text{ €/h} = 3.375 \text{ €}$

Adicionalmente, en un entorno profesional podrían considerarse ciertos costes asociados a herramientas de software utilizadas durante el desarrollo del proyecto. Por ejemplo, para la creación y edición de gráficos en pixel art podría utilizarse la herramienta Aseprite, cuyo coste aproximado es de 16,79 €. No obstante, en el presente proyecto se han utilizado alternativas gratuitas o licencias académicas equivalentes.

Asimismo, el desarrollo se ha realizado utilizando un equipo informático personal valorado aproximadamente en 1.200 €. Considerando una vida útil estimada del equipo de cuatro años y un periodo de desarrollo aproximado de dos meses, el coste imputable al proyecto mediante amortización sería de aproximadamente 50 €.

Teniendo en cuenta estos factores, el coste total estimado del proyecto en un contexto profesional sería aproximadamente:

- Coste de desarrollo: 3.375 €
- Licencias de software: 16,79 €
- Amortización de hardware: 50 €

Coste total estimado del proyecto: 3.441,79 €

Análisis

6.1. Actores y roles

6.2. Modelado de Requisitos

Los requisitos describen las condiciones, capacidades y restricciones que debe cumplir un sistema software para satisfacer las necesidades de sus usuarios y los objetivos de la organización.

Se distinguen principalmente tres tipos de requisitos: los requisitos funcionales (RF), que especifican los servicios y comportamientos que debe ofrecer el sistema; los requisitos no funcionales (RNF), que establecen restricciones y propiedades de calidad como rendimiento, fiabilidad o usabilidad; y los requisitos de información (RI), que definen los datos que el sistema debe almacenar y gestionar para proporcionar dichos servicios.

Lista de Requisitos Funcionales

- RF01: El sistema permitirá iniciar una nueva partida desde un estado inicial controlado, inicializando el tablero, las piezas y el turno inicial conforme a las reglas del juego.
- RF02: El sistema permitirá al jugador colocar sus piezas al inicio de la partida dentro de la zona de despliegue definida.
- RF03: El sistema gestionará los turnos alternos entre el jugador y el bot, impidiendo la realización de acciones fuera de turno.
- RF04: El sistema validará las acciones realizadas por los jugadores, evitando movimientos o ataques no permitidos por las reglas del juego.
- RF05: El sistema revelará la identidad de las piezas implicadas cuando se produzca un combate, manteniendo visible la información revelada durante el resto de la partida.

- RF06: El bot tomará decisiones de movimiento y ataque de forma autónoma, respetando las reglas y restricciones del juego.
- RF07: El sistema mantendrá y actualizará el estado de la partida durante su desarrollo, gestionando turnos, acciones válidas y transiciones entre estados.
- RF08: El sistema determinará automáticamente el final de la partida cuando se cumpla una condición de victoria o derrota, mostrando el resultado al jugador.
- RF09: El sistema permitirá activar o desactivar mecánicas opcionales, como refuerzos o ventaja de ataque, antes del inicio de la partida.
- RF10: La interfaz mostrará información visual y sonora relacionada con las acciones realizadas, los combates y los cambios de estado de la partida.
- RF11: La interfaz del juego deberá ser completamente manejable mediante ratón.
- RF12: El sistema permitirá al jugador abandonar la partida en curso de forma controlada.

Requisitos no funcionales

- RNF01: El sistema deberá ser compatible con sistemas operativos Windows 10 y Windows 11.
- RNF02: El videojuego deberá ejecutarse correctamente en un equipo con las siguientes especificaciones mínimas de hardware:
 - Procesador: CPU de doble núcleo a 2.0 GHz o superior.
 - Memoria RAM: 8 GB.
 - Tarjeta gráfica: GPU compatible con OpenGL 3.3 o superior.
 - Almacenamiento: ...

Lista de Requisitos Funcionales de Información

- RI01: Juego, el sistema gestionará una sesión de juego tratando la siguiente información de una partida:
 - Lista de casillas que conforman el tablero.
 - Lista de todas las piezas.
 - Turno actual (Jugador o Bot)
- RI02: Casillas, las cuales tendrán las siguientes propiedades:
 - Posición (x, y).
 - Tipo (No transitable, transitable, zona de despliegue para el jugador o el bot).

- Referencia a la pieza que la esta ocupando.
- RI03: Piezas, de las cuales se deberán saber las siguientes características:
 - Nombre
 - Rango
 - Propietario (Jugador o Bot)
 - Estado (Oculto, revelado o eliminado)

6.3. Casos de Uso

6.4. Modelo de dominio

6.5. Modelo de proceso de negocio

Diseño

- 7.1. Arquitectura del sistema
- 7.2. Módulos o Componentes del sistema
- 7.3. Diseño de clases
- 7.4. Diseño de interfaz de usuario
- 7.5. Patrones de diseño aplicados
- 7.6. Seguridad y rendimiento

Implementación

8.1. Organización del código

8.2. Licencias requeridas

8.3. Batería de pruebas

Capítulo 9

Conclusiones y Líneas futuras

Apéndice A

Manual de instalación

Apéndice B

Manual de usuario

Apéndice C

Apéndice D

Bibliografía

- [1] J. M. Sadurní. El juego real de ur, un entretenimiento en la antigua mesopotamia. https://historia.nationalgeographic.com.es/a/el-juego-real-de-ur-un-entretenimiento-en-la-antigua-mesopotamia_19167.
- [2] Brooklyn Museum. Gaming board inscribed for amenhotep iii with separate sliding drawer. https://web.archive.org/web/20160307212542/https://www.brooklynmuseum.org/opencollection/objects/3536/Gaming_Board_Inscribed_for_Amenhotep_III_with_Separate_Sliding_Drawer.
- [3] Chess Variants. Dou shou qi (animal chess). <https://www.chessvariants.org/other.dir/animal.html>.
- [4] British Antique Dealers' Association. L'attaque board. <https://collections.vam.ac.uk/item/O26365/lattaque-the-famous-game-of-board-game/>.
- [5] Ancient Chess. How to play the chinese land battle game luzhanqi. <https://ancientchess.com/page/play-luzhanqi.htm>.
- [6] RocketBrush Studio. Godot vs. unity vs. unreal comparison. <https://rocketbrush.com/blog/godot-vs-unity-vs-unreal-comparison>.
- [7] Proyectos Agiles. Desarrollo iterativo e incremental. <https://proyectosagiles.org/desarrollo-iterativo-incremental/>.
- [8] British Museum. The royal game of ur. https://www.britishmuseum.org/collection/object/W_1928-1009-378.
- [9] Danielle Zwang. Senet and twenty squares: Two board games played by ancient egyptians. <https://www.metmuseum.org/es/perspectives/ancient-egypt-board-games>.
- [10] Colin Stapczynski. History of chess | from early stages to magnus. <https://www.chess.com/article/view/history-of-chess>.
- [11] British Antique Dealers' Association. L'attaque game. <https://www.bada.org/object/lattaque-game>.
- [12] History of stratego. <https://www.ultraboardgames.com/stratego/history.php>.